Unit II – Linear Data Structures Using Sequential Organization

Prof. Pranjal Pandit

Unit II – Linear Data Structures Using Sequential Organization

- Concept of sequential organization;
- Arrays as an ADT and array operations;
- merging of two arrays; storage representation and address calculation (row-major, column-major);
- multi-dimensional arrays.
- Ordered lists;
- Single-variable polynomial representation using arrays, polynomial as array of structures, polynomial addition and multiplication.
- Sparse matrices: representation, addition, simple and fast transpose, time–space trade-off.

Sparse Matrices: Representation, Addition, Simple Transpose, Fast Transpose, and Time–Space Trade-off

A **matrix** is a rectangular arrangement of numbers in rows and columns.

Example:

A =

1 2 3

4 5 6

7 8 9

What is a Sparse Matrix?

A **sparse matrix** is a matrix in which **most of the elements are zero**.

Definition

A sparse matrix is a matrix containing **more zero elements than non-zero elements**.

Example

A =

0 0 0 5

0 8 0 0

0 0 0 0

3 0 0 0

Total elements

4 × 4 = 16

Non-zero elements 3

Zero elements 13

Dense Matrix vs Sparse Matrix

Dense Matrix

1 2 3

4 5 6

7 8 9

Almost every element is non-zero.

Why Do We Use Sparse Matrix Representation?

Suppose we have a matrix

$$1000 \times 1000$$

Total elements 1000000

Suppose only **20 elements are non-zero**.

If stored normally, Need space for 1,000,000 integers

Most memory stores only zeros. This is a waste of memory.

Instead, we store only the **non-zero elements**

Sparse Matrix Representation (3-Tuple Representation)

The most common representation is the **3-Tuple (Triplet) Representation**.

Each non-zero element is stored as:

(Row, Column, Value)

The **first row** stores

(Number of Rows,

Number of Columns,

Number of Non-zero Elements)

Example 1

Matrix

0 0 3

22 0 0

0 0 5

Step 1

Rows = 3

Columns = 3

Non-zero elements = 3

Step 2

Write row, column and value

Row	Column	Value
0	2	3
1	0	22
2	2	5

Final Representation

Row	Column	Value
3	3	3
0	2	3
1	0	22
2	2	5

Example 2

Matrix

```
0  4  0  0
0  0  8  0
5  0  0  0
```

Non-zero elements

```
4  8  5
```

Triplet Representation

Row	Column	Value
3	4	3
0	1	4
1	2	8
2	0	5

Sparse Matrix Addition

Two matrices can be added only if:

- Number of rows are equal.
- Number of columns are equal.
- Addition is performed by adding elements at the same position.

Example

Matrix A

0 0 5

3 0 0

0 7 0

Matrix B

0 2 0

4 0 0

0 1 6

Add

0+0 =0

0+2 =2

5+0 =5

3+4 =7

7+1 =8

0+6 =6

Result

0 2 5

7 0 0

0 8 6

Addition Using Triplets

Matrix A

Row	Column	Value
3	3	3
0	2	5
1	0	3
2	1	7

Matrix B

Row	Column	Value
3	3	4
0	1	2
1	0	4
2	1	1
2	2	6

Different positions are copied directly. Final triplet

Row	Column	Value
3	3	5
0	1	2
0	2	5
1	0	7
2	1	8
2	2	6

Matrix Transpose

Transpose means

Convert every row into a column.

Notation

$$A^T$$

Example

Original Matrix

1 2 3

4 5 6

Transpose

1 4

2 5

3 6

Rows become columns.

Simple Transpose of Sparse Matrix

In triplet representation,
Each element
`(Row, Column, Value)`
becomes
`(Column, Row, Value)`

Example
Original Triplet

Row	Column	Value
3	3	3
0	2	5
1	0	7
2	1	8

Swap row and column

Row	Column	Value
3	3	3
2	0	5
0	1	7
1	2	8

Original Matrix

$$\begin{bmatrix} 0 & 0 & 3 & 0 \\ 22 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 7 & 0 & 5 \end{bmatrix}$$

3-Tuple representation

Row	Col	Value
4	4	4
0	2	3
1	0	22
3	1	7
3	3	5

Step 1

Copy first row by swapping rows and columns.

Original

4 4 4

Transpose

4 4 4

(No change because rows = columns.)

Step 2

Now scan **column by column**.

Columns are

0

1

2

3

Column 0

Search every row whose column = 0

Found 1 0 22

Swap row and column 0 1 22

Output becomes

4 4 4

0 1 22

Column 1

Search entries having column =1

Found 3 1 7

Swap 1 3 7

Output

4 4 4

0 1 22

1 3 7

Row	Col	Value
4	4	4
0	2	3
1	0	22
3	1	7
3	3	5

Column 2

Found 0 2 3

Swap 2 0 3

Output

4 4 4

0 1 22

1 3 7

2 0 3

Column 3

Found 3 3 5

Swap 3 3 5

Row	Col	Value
4	4	4
0	2	3
1	0	22
3	1	7
3	3	5

Final output

Row	Col	Value
4	4	4
0	1	22
1	3	7
2	0	3
3	3	5

Algorithm (Simple Transpose)

Copy first row.

For each column c

    For every non-zero element

        If column == c

            Swap row and column

            Store in transpose

Time Complexity

Outer loop = columns = **n**

Inner loop = non-zero elements = **t**

Therefore,

$$O(n \times t)$$

If there are many columns, this becomes slow.

Fast Transpose

Simple transpose repeatedly scans the entire array for each column. Fast transpose avoids this by computing exactly where each transposed element belongs.

The idea has **three main steps**.

Example

Original 3-Tuple

There are **4 columns** (0–3).

Step 1: Count how many elements are in each column

Look at the **column values**:

2

0

1

3

Row	Col	Value
4	4	4
0	2	3
1	0	22
3	1	7
3	3	5

Count occurrences:

Column	Count
0	1
1	1
2	1
3	1

Store in an array:

rowTerms = [1,1,1,1]

If column 2 had three elements, then

rowTerms[2]=3

Step 2: Find the starting position of each transposed row

Create another array called startingPos. The first non-header entry starts at position **1**.

startingPos[0]=1

Now calculate the remaining positions:

startingPos[1]

= startingPos[0] + rowTerms[0]

=1+1=2

startingPos[2]

=2+1=3

startingPos[3]

=3+1=4

So

startingPos = [1,2,3,4]

This tells us where each transposed row begins.

Column	Starting Position
0	1
1	2
2	3
3	4

Step 3: Place each element directly

Read each original entry once.

Entry

0 2 3

Column =2

Place at

startingPos[2]=3

Output row 3

2 0 3

Increment

startingPos[2]++

Now

startingPos=[1,2,4,4]

Entry

1 0 22

Column =0

Place at row 1

0 1 22

Increase

startingPos[0]=2

Entry

3 1 7

Column =1

Place at row2

1 3 7

Increase

startingPos[1]=3

Entry

3 3 5

Column =3

Place at row4

3 3 5

Final Result

This is the same result as the simple transpose, but it is produced much more efficiently.

Row	Col	Value
4	4	4
0	2	3
1	0	22
3	1	7
3	3	5

Column	Starting Position
0	1
1	2
2	3
3	4

Row	Col	Value
4	4	4
0	1	22
1	3	7
2	0	3
3	3	5

Why Is It Called "Fast"?

Instead of scanning all entries once for every column, fast transpose:

- Counts entries in each column.
- Computes the starting position for each transposed row.
- Places every element directly into its correct position.
- Each non-zero element is processed only once after the counting step.

Code

```
#include <iostream>
using namespace std;

struct Term {
    int row, col, value;
};

int main ()
{
    int choice;
    int rows, cols;
    int matrix[20][20];
    Term sparse[100];
    int nonZero=0;
    bool sparseCreated=false;
```

```
do{
        cout<<"\n========== MENU ==========";
        cout<<"\n1. Row Major Address";
        cout<<"\n2. Column Major Address";
        cout<<"\n3. Create Sparse Matrix";
        cout<<"\n4. Simple Transpose";
        cout<<"\n5. Fast Transpose";
        cout<<"\n6. Exit";
        cout<<"\nEnter Choice: ";
        cin>>choice;

        switch(choice)
```

```
case 1: ( Row Major Address)
    {
        int r,c,base,size;
        cout<<"Enter rows and cols: ";
        cin>>rows>>cols;
        cout<<"Enter Base Address: ";
        cin>>base;
        cout<<"Enter Size: ";
        cin>>size;
        cout<<"Enter row and column index: ";
        cin>>r>>c;
        if(r<0||r>=rows||c<0||c>=cols)
            cout<<"Invalid Index";
        else
            cout<<"Row Major Address = "<<base+((r*cols)+c)*size;
        break;
    }
```

```
case 2: (Column Major Address)
        {
            int r,c,base,size;
            cout<<"Enter rows and cols: ";
            cin>>rows>>cols;
            cout<<"Enter Base Address: ";
            cin>>base;
            cout<<"Enter Size: ";
            cin>>size;
            cout<<"Enter row and column index: ";
            cin>>r>>c;
            if(r<0||r>=rows||c<0||c>=cols)
                cout<<"Invalid Index";
            else
                cout<<"Column Major Address = "<<base+((c*rows)+r)*size;
            break;
        }
```

```
case 3: (Create Sparse Matrix)
      {
         cout<<"Enter rows and cols: ";
         cin>>rows>>cols;
         cout<<"Enter matrix:\n";
         for(int i=0;i<rows;i++)
            for(int j=0;j<cols;j++)
               cin>>matrix[i][j];

         int k=1;
         sparse[0].row=rows;   (sparse is a struct term- creating first row)
         sparse[0].col=cols;
         sparse[0].value=0;
```

```
for(int i=0;i<rows;i++)                    (Creating 3 tuple representation)
        for(int j=0;j<cols;j++)
           if(matrix[i][j]!=0)            (For non zero elements)
        {
              sparse[k].row=i;
              sparse[k].col=j;
              sparse[k].value=matrix[i][j];
              k++;
           }

    sparse[0].value=k-1;  (Adding total no of non zero value to sparse[0].value)
    nonZero=k-1;
    sparseCreated=true;

    cout<<"\nSparse Matrix\n";
    cout<<"Row\tCol\tValue\n";
    for(int i=0;i<=nonZero;i++)
       cout<<sparse[i].row<<"\t"<<sparse[i].col<<"\t"<<sparse[i].value<<endl;
    break;
  }
```

```
case 4: (Simple Transpose)
      {
         if(!sparseCreated){
            cout<<"Create sparse matrix first.";
            break;
         }
         Term transpose[100];                          //initialize 1st row
         transpose[0].row=sparse[0].col;
         transpose[0].col=sparse[0].row;
         transpose[0].value=sparse[0].value;
         int k=1;
         for(int col=0;col<sparse[0].col;col++)
            for(int i=1;i<=nonZero;i++)

              if(sparse[i].col==col){                          (For non zero values swap rows and column)
                  transpose[k].row=sparse[i].col;
                  transpose[k].col=sparse[i].row;
                  transpose[k].value=sparse[i].value;
                  k++;
               }
```

```
 cout<<"\nSimple Transpose\n";
        cout<<"Row\tCol\tValue\n";
        for(int i=0;i<=nonZero;i++)

cout<<transpose[i].row<<"\t"<<transpose[i].col<<"\t"<<transpose[i].val
ue<<endl;
        break;
      }
```

```
case 5: (Fast Transpose)
      {
         if(!sparseCreated){
            cout<<"Create sparse matrix first.";
            break;
         }
         Term fast[100];
         int rowTerms[20]={0}, startPos[20]={0};

         fast[0].row=sparse[0].col;              (create the first row)
         fast[0].col=sparse[0].row;
         fast[0].value=sparse[0].value;

         for(int i=1;i<=nonZero;i++)        (Count the non zero-column values)
            rowTerms[sparse[i].col]++;

         startPos[0]=1;
         for(int i=1;i<sparse[0].col;i++)
            startPos[i]=startPos[i-1]+rowTerms[i-1];       (create starting position array)
```

```
 for(int i=1;i<=nonZero;i++)
// (Swap rows and column as per the starting position)
{
        int pos=startPos[sparse[i].col];
        fast[pos].row=sparse[i].col;
        fast[pos].col=sparse[i].row;
        fast[pos].value=sparse[i].value;
        startPos[sparse[i].col]++;
    }
```

```
   cout<<"\nFast Transpose\n";
    cout<<"Row\tCol\tValue\n";
    for(int i=0;i<=nonZero;i++)
        cout<<fast[i].row<<"\t"<<fast[i].col<<"\t"<<fast[i].value<<endl;
    break;
}

case 6:
    cout<<"Program Ended";
    break;
```

Example for practice

0	0	3	0	4
0	0	5	7	0
0	0	0	0	0
0	2	6	0	0

For this Sparse matrix- create 3 tuple representation, perform simple transform and fast transform operations on it.